

ユニット 9・第15-16週

AI ネイティブ OS : OS の 中のエージェントと LLM

📅 第15-16週・日程は未定 📖 演習 TD8 (解答 + 生成AIチャレンジ) 🎬 アニメーション 4 本

マニュアル M9・ユニットの理論

AI ネイティブ OS : OS 中のエージェントと LLM

ユニット 9 の基本の読み物 (約 1.5 時間)。このユニットはコース全体をまとめます: AI エージェントはシステムコール、ファイル、メモリ、スレッド、ソケットを使うプロセスの 1 つにすぎませんが、その判断は誤りうるモデルが下します。ここで挙げる製品は変化が速いので、主張にはすべて出典と日付を付けています (2026/09/25 確認)。

目次

- | | | | |
|----|-------------------------|----|-----------------|
| §1 | チャットからエージェントへ | §4 | Linux の隔離プリミティブ |
| §2 | Windows と Linux が発表したこと | §5 | モデルが提案し、OS が決める |
| §3 | OS の負荷としての推論 | | |

§1 チャットからエージェントへ

言語モデル (LLM) はテキストを受け取り、次のテキストを予測します。エージェントはそこにループを加えます: モデルが行動 (`read_file("notas.txt")`) のようなツール呼び出しを提案し、プログラムがそれを実行し、結果がモデルに戻ります。こうしてアシスタントはファイルを整理したり、設定を変えたり、アプリケーションに入力したりできます。

オペレーティングシステムにとって、これは 2 つの理由で新しいものです: エージェントは**ユーザの代わりに動くがユーザ本人ではない**こと、そしてその判断が、誤ったり操られたりしうる統計モデルから来ること (たとえば読んだ文書に隠された指示によるプロンプトインジェクション)。だからこのユニットの問いはオペレーティングシステムの問いです: エージェントとは誰か? 何に触れてよいか? 誰が承認するか? どう監査するか?

§2 Windows と Linux が発表したこと

Windows

- **Agent workspace**: エージェントが**専用のアカウント**でユーザと並行して働く別の空間。既知のフォルダ (ドキュメント、ダウンロード、デスクトップ、ミュージック、ピクチャ、ビデオ) には、ユーザが有効にして同意した場合にだけアクセスする。設定 → システム → AI コンポーネント → 試験的なエージェント機能で有効にする (プレビュー)。出典: Microsoft Support 「Experimental agentic features」。

- **Agent connectors**：エージェントをアプリケーションやシステムのツールにつなぐ *Model Context Protocol* (MCP) サーバで、**On-Device Registry** に登録される。最初のはエクスプローラーと設定向け (Insiders、2025 年 12 月)。出典：Microsoft Support、The Register 2025/12/08。
- 宣言された原則：エージェントの行動はユーザの行動と**区別して観察できる**こと、データと行動へのユーザの**承認**、そして**最小権限** (起動したユーザより多くの権限は決して持たない)。
- **Microsoft Execution Containers** (Build 2026、2026 年 6 月 2 日)：Windows と WSL 上のエージェントのための、宣言的なポリシーを持つ実行層。プロセス単位、セッション単位、またはハイパーバイザに支えられたマイクロ VM での隔離。初期プレビュー段階。出典：Windows Developer Blog 「Windows platform security for AI agents」 2026/06/02。

Linux

- 唯一の「システムのエージェント」はない：**カーネルがプリミティブを提供し** (§4)、ディストリビューションとアプリケーションがそれを組み合わせる。
- **Canonical (Ubuntu)** は 2026 年に、オープンなモデルによるローカル推論 (*inference snaps*) と、snap の仕組みで隔離したエージェントを中心とする AI 機能を発表した。最初のプレビューは Ubuntu 26.10 で見込まれている。AI 全体のスイッチはなく、snap を削除して無効にする。出典：OMG! Ubuntu 2026/04、Phoronix。
- AI アクセラレータについて、Linux には **accel** サブシステム (Intel と AMD の NPU ドライバ) がある。カーネルのコミュニティでは 2025 年 12 月から、AI ツールの助けを **Assisted-by** タグで申告するようドキュメントが求めている。**Signed-off-by** の署名を加えられるのは人間だけ。

実践 → アニメーション A9.1・エージェント型 OS のアーキテクチャ

§3 OS の負荷としての推論

オペレーティングシステムにとって、モデルの実行はとても特徴的なプロファイルを持つプログラムです：

- **巨大で読み取り専用の重み**。**mmap** で読み込みます (ユニット 6)：コピーせず、触れたときにページキャッシュから読み (ページフォールト)、複数のプロセスが同じ物理ページを共有します。TD8 の micro-LLM は約 0.7 MB の重みをこうしてマップします。
- **KV キャッシュ**。処理済みの各トークンについて、モデルは計算し直さないよう各層のキーと値を保持します：会話の長さとともに大きくなるメモリです (TD8 では $2 \times \text{層数} \times \text{コンテキスト} \times 64 \times 4$ バイト)。
- **2つのフェーズ**。*prefill* は要求全体を一度に処理し、*decode* は 1 ステップに 1 トークンずつ生成し、そのたびに重みをすべて読みます。
- **アクセラレータ**。GPU や NPU はデバイスとして扱います：ドライバ、専用メモリ、プロセス間のアクセスのスケジューリング。

実践 → アニメーション A9.2・micro-LLM の内側

§4 Linux の隔離プリミティブ

プリミティブ	制限するもの	ROOT が必要か
ユーザと権限	所有者、グループ、モードに応じたファイル	ユーザを作るとき
Landlock (5.13 以降)	プロセスが開けるファイルシステムのパス、TCP ポート (ABI 4)、範囲外のシグナルと抽象ソケット (ABI 6)	不要 ：プロセスが自分自身を制限する
seccomp	行えるシステムコール	不要 (<code>no_new_privs</code> 付き)
Namespaces	見えるもの：プロセス、マウント、ネットワーク、ユーザ (コンテナの土台)	ユーザ namespace なら不要
cgroups	消費量：CPU、メモリ、入出力	たいてい必要 (または systemd が委譲)
<code>setrlimit</code>	プロセスごとの上限：CPU 時間、ファイルサイズ、ディスクリプタ	下げるだけなら不要

TD8 は `agent_exec` で Landlock、`setrlimit`、`alarm` を使います：`landlock_restrict_self` の後は、たとえ自分のコードが試みても、プロセスは作業領域から出られません。この制限は受け継がれ、取り消せません。

§5 モデルが提案し、OS が決める

このユニットの中心となる教訓：**モデルの出力は信用できないテキスト**。TD8 のエージェント型 OS はそれを他の外部入力と同じように扱います：

1. **厳密なパーサ**：閉じたカタログの `name("arg", "arg")` だけを受け付け、ほかはすべて拒否する。
2. **ポリシー**：作業領域内の相対パスのみ、`..` なし、シンボリックリンクを解決する (`realpath`)。
3. 破壊的なものにはユーザの同意。
4. 最後の砦としての**カーネルのサンドボックス** (Landlock)：ポリシーが失敗しても無効でも、カーネルはアクセスを拒否する。
5. **監査**：すべての判断がクライアントの uid と pid (`S0_PEERCREC`) とともに `audit.log` に記録される。これが**多層防御**の原則です：どの層も、前の層が失敗しなかったと仮定しません。

実践 → アニメーション A9.4・モデルが提案し、OS が決める、そして TD8